



4th Exercise

Architecture and CAD for FPGAs

Context

Computing the routing on an FPGA is compute intensive and contributing substantially to the time for implementing a bitstream. While finding a single path is not a big deal, keep in mind that large FPGAs have more than a million LUTs and tens of millions of wires. Moreover, we have to compute the negotiation, which is deciding which physical routing resource will be used for implementing which specific net (negotiation is not part of this exercise). This is why we dedicate more exercises to path search algorithms. In our last exercise, you implemented a Breadth-First Search (BFS) for finding a path between some primitive pins. In this exercise, we want to rerun those experiments and use the shortest path algorithm from Dijkstra and its variant A*.

Aufgabe 1 (Dijkstra's Algorithm)

In this task, we want to repeat the experiments from our last exercise by using Dijkstra's algorithm. If you never implemented that before, you will find plenty of material on this online. For instance, Wikipedia is brilliant:

<https://de.wikipedia.org/wiki/Dijkstra-Algorithmus> or

<https://www.youtube.com/watch?v=KiOso3VE-vI>.

(The latter is nice, but should have marked the nodes visited.)

- a) Dijkstra's algorithm uses non-negative weights annotated on the edges. Discuss briefly good weights that represent the problem of FPGA routing. In general, the cost of using an edge could be a cost function and you can discuss that too.
- b) Implement Dijkstra's algorithm and run the experiments as in Exercise 3 (the BFS one). As before, instrument your code to record 1) the number lookups in your dictionary and 2) the size of your search front (e.g., the entries of your job queue for your version of Dijkstra's algorithm).

Aufgabe 2 (A* Algorithm)

In Dijkstra's algorithm, we maintain a job queue with the nodes to visit and we expand the search by visiting the node with the cheapest cost (or a random node if multiple ones have the cheapest cost). However, that could result in expanding the search front in opposite direction to the target (which may be the right way in some cases but which is bad in most cases). A* for FPGA routing tries to converge faster by expanding to a node that leads closer to the target. Because we do not route diagonal, the Manhattan distance is a good metric to select for the expansion. For instance, you can expand your search front in the direction (north/east/south/west) that minimizes the distance to the target most. Note that the cost is not allowed to be an overestimate in order to guarantee that A* converges.

- a) Keep your Dijkstra's algorithm and modify it in a new variant to implement A*. As they are very similar, you may use a switch to run your search in Dijkstra or A* mode. You may have to think about your model (or dictionary) in order to facilitate the A* expansion. As before, instrument your code to record 1) the number lookups in your dictionary and 2) the size of your search front.

- b) Record when you reached the target node the first time and how the cost reduced after that. (Basically a plot or table of the cost to reach the destination over the iteration count.) Propose a heuristic to trade execution time versus quality of the search. As quality here, we count the number of wires used. Other metrics could be the number of routing tracks used or latency. (The latter would require a timing model that we haven't provided to you. However, if you want to play with timing, you can use a constant initial cost c_i plus a track cost c_t as a good estimate. For instance, with $c_i = 50 ps$ and $c_t = 20 ps$, a jump/single/quad wire would cost $50 ps / 70 ps / 130 ps$.) Please note that modeling physical resource cost and latency cost are related but different things.

Aufgabe 3 (Minimum Spanning Trees (optional))

Both Dijkstra's algorithm and A* can compute a shortest path to multiple destinations (i.e. a spanning tree). Note that a lookup table has more inputs than outputs. This implies that most nets have a fanout that is larger than 1 and spanning trees are the normal case.

However, routing spanning trees may use an excessive amount of routing resources and we are usually interested in trees that minimize our scarce routing resources (i.e. minimum (or minimized) spanning trees). See for instance, see Figure 1 in the paper *CRoute: A Fast High-quality Timing-driven Connection-based FPGA Router*, which you can find in Moodle.

- a) Read about minimum spanning trees and modify your path search to implement spanning trees that use less routing resources. As an experiment on our 8x8 CLB fabric, route from one top left ALUT.0 LUT output pin to the [A-H] LUT.I0 input pins on the diagonal bottom-left to top-right ($8 \times 8 = 64$ LUT input destinations).

Compare your results with routing methods that do not optimize spanning trees and report your findings.

Aufgabe 4 (Obstacles (optional))

It is not unusual that FPGA fabrics contain obstacles (e.g., a CPU block in the middle or the corner of the fabric). Think of how your different path search algorithms behave in case of obstacles. Run experiments to get a feeling for the impact of obstacles. You can either do an inverse grep on your model and filter out some wires to create some obstacles or you modify your fabric CSV in FABulous. Report your findings.

Zip your code and a short report containing the findings and answers for both **exercise 3 and 4** and submit that to Moodle.